OPERATING SYSTEMS (UE24CS242B)
ORANGE PROBLEM

NAME : AADHAVAN MUTHUSAMY
SRN : PES1UG24CS002
SECTION : CSE - 4A

Phase 1: Object Storage Foundation

Screenshot 1A: Output of `./test_objects` showing all tests passing.

```
gcc -o test_objects test_objects.o object.o -lcrypto
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$
```

Screenshot 1B: `find .pes/objects -type f` showing the sharded directory structure.

```
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$
```

Phase 2: Tree Objects

Screenshot 2A: Output of `./test_tree` showing all tests passing.

```
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from `find .pes/objects -type f` and run `xxd .pes/objects/XX/YYY... | head -20` to show the raw binary format.

```
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$
```

Phase 3: The Index (Staging Area)

Screenshot 3A: Run `./pes init`, `./pes add file1.txt file2.txt`, `./pes status` — show the output.

```
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ echo "hello" > file1.txt
echo "world" > file2.txt
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ ./pes add file1.txt file2.txt
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  commit.c
  untracked:  object.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  index.c
  untracked:  tree.h
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  test_objects.c
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_tree
  untracked:  index.h
  untracked:  Makefile
```

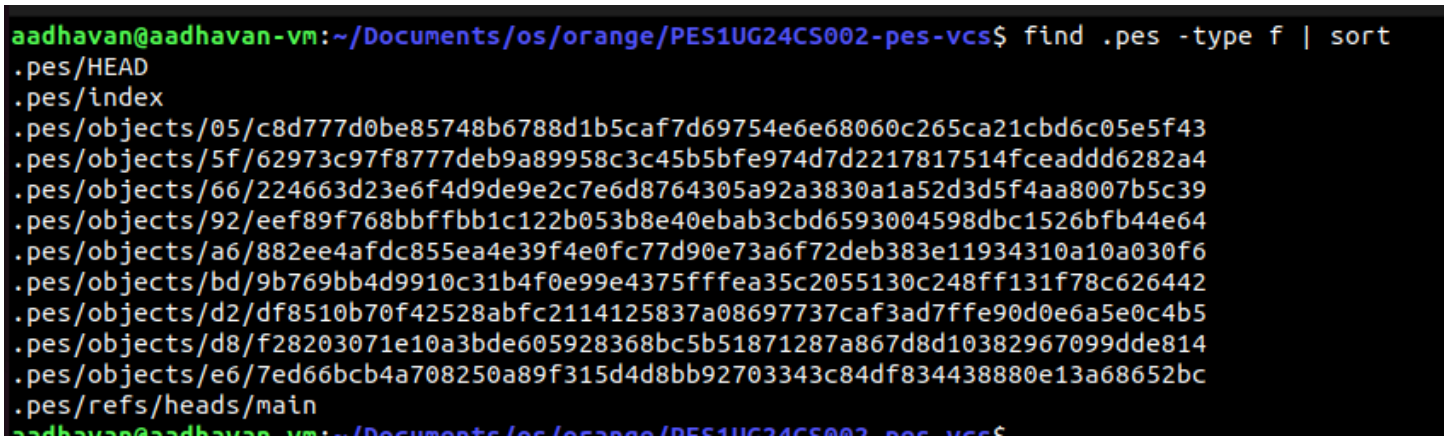
Screenshot 3B: `cat .pes/index` showing the text-format index with your entries.

```
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776670986 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776670986 6 file2.txt
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$
```

Phase 4: Commits and History

Screenshot 4A: Output of `./pes log` showing three commits with hashes, authors, timestamps, and messages.

Screenshot 4B: `find .pes -type f | sort` showing object store growth after three commits.



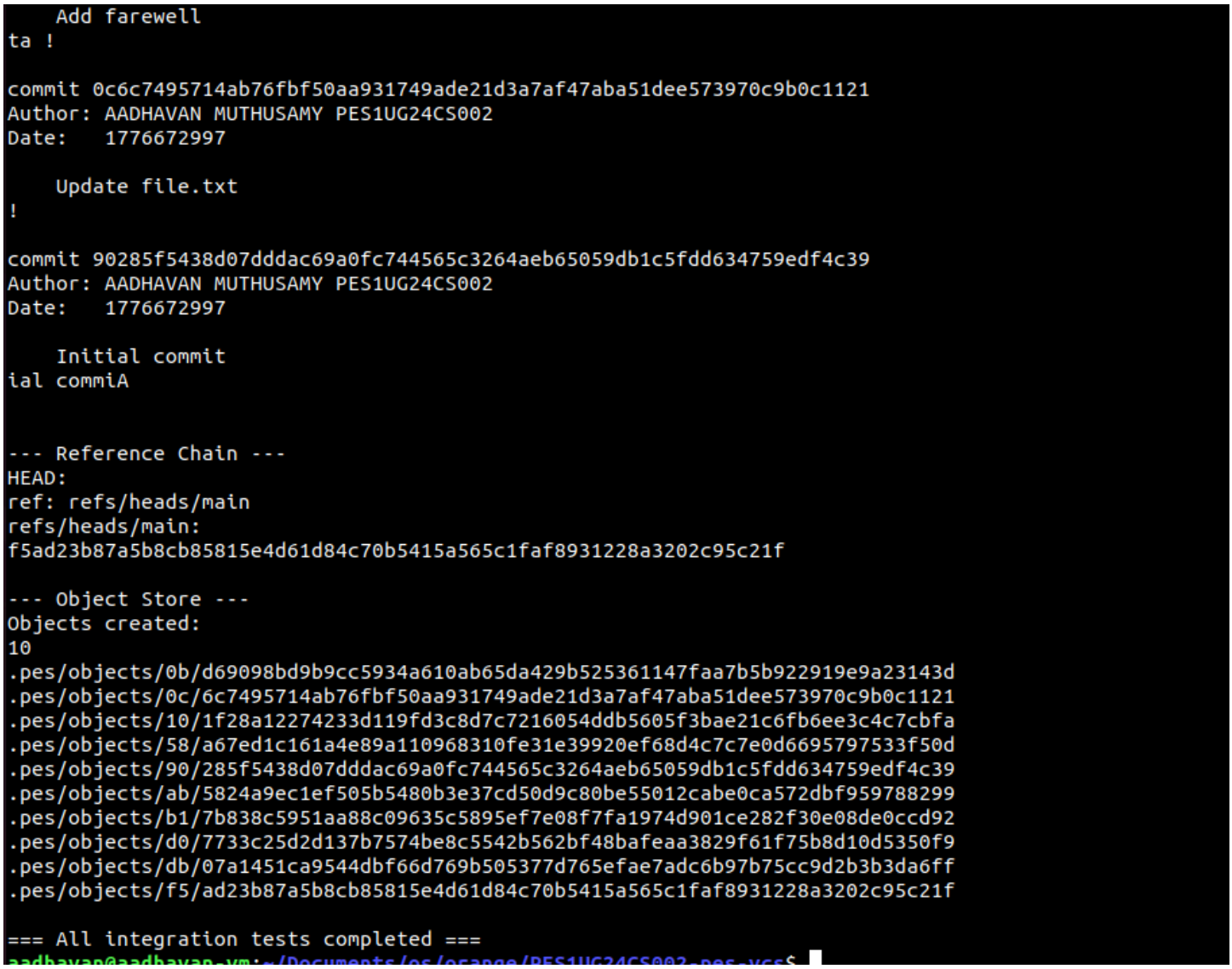
Screenshot 4C: `cat .pes/refs/heads/main` and `cat .pes/HEAD` showing the reference chain.

```
.pes/refs/heads/main
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ cat .pes/refs/heads/main
5f62973c97f8777deb9a89958c3c45b5bfe974d7d2217817514fceaddd6282a4
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$
```

Full integration test (`make test-integration`)

```
aadhavan@aadhavan-vm:~/Documents/os/orange/PES1UG24CS002-pes-vcs$ make test-integration
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
commit.c: In function ‘commit_create’:
commit.c:204:5: warning: implicit declaration of function ‘mkdir’ [-Wimplicit-function-declaration]
  204 |     mkdir(".pes/refs", 0755);
      |     ^~~~~
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

```
--- First Commit ---
Committed: 90285f5438d0... Initial commit

Log after first commit:
commit 90285f5438d07dddac69a0fc744565c3264aeb65059db1c5fdd634759edf4c39
Author: AADHAVAN MUTHUSAMY PES1UG24CS002
Date:   1776672997

    Initial commit



--- Second Commit ---
Committed: 0c6c7495714a... Update file.txt

--- Third Commit ---
Committed: f5ad23b87a5b... Add farewell

--- Full History ---
commit f5ad23b87a5b8cb85815e4d61d84c70b5415a565c1faf8931228a3202c95c21f
Author: AADHAVAN MUTHUSAMY PES1UG24CS002
Date:   1776672997

    Add farewell
ta !

commit 0c6c7495714ab76fbf50aa931749ade21d3a7af47aba51dee573970c9b0c1121
Author: AADHAVAN MUTHUSAMY PES1UG24CS002
Date:   1776672997

    Update file.txt
!

commit 90285f5438d07dddac69a0fc744565c3264aeb65059db1c5fdd634759edf4c39
Author: AADHAVAN MUTHUSAMY PES1UG24CS002
Date:   1776672997

    Initial commit
ial commit
```



Phase 5 & 6: Analysis-Only Questions

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

A5.1: Implementing **`pes checkout`**

- **`.pes/`** Changes**:** `.pes/HEAD` is updated to point to the new branch (e.g., `ref: refs/heads/new-branch`). The `.pes/index` file must be completely rewritten to match the tree of the target branch's latest commit.
- Working Directory**:** Files must be added, deleted, or overwritten to exactly mirror the target branch's snapshot.

- Complexity**:** The difficulty lies in doing this safely. You have to ensure you don't accidentally overwrite a user's uncommitted work while swapping out the underlying filesystem state, requiring careful diffing before touching any files.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A5.2: To detect this safely using only the index and object store:

- Compute the hash of the file currently in the working directory.
- Compare it against the hash stored for that file in the `.pes/index`. If they differ, the file is "dirty" (modified but uncommitted).
- Next, look up that same file's hash in the target branch's root tree (in the object store).
- If the target tree's hash differs from the index's hash, you have a conflict. The checkout must abort because switching branches would require overwriting the user's unsaved changes to make the file match the target branch.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

A5.3: Detached HEAD State

- What happens**:** When you make a commit in a detached HEAD, the new commit object is created, and `.pes/HEAD` is updated to contain the new raw commit hash directly. The commit chain is valid, but no branch file in `refs/heads/` was updated to track it.
- Recovery**:** If you switch branches, those commits become "orphaned" and practically invisible. To recover them, you simply create a new file in `.pes/refs/heads/` (a new branch) and write the hash of the last detached commit into it.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

A6.1: Garbage Collection Algorithm

- Algorithm (Mark and Sweep)**:** 1. **Mark:** Iterate through all branch files in `.pes/refs/heads/` and `.pes/HEAD`. Walk their commit histories backward. For every commit visited, parse its tree and recursively mark the commit, tree, and all associated blobs as "reachable." 2. **Sweep:** Traverse the `.pes/objects/` directory. If you find an object hash on disk that wasn't marked, delete it.
- Data Structure**:** A Hash Set (or a Bloom filter paired with a hash set for heavy loads) is ideal here for O(1) lookups to quickly check if a hash has already been marked.
- Estimation**:** For 100,000 commits, you must visit exactly 100,000 commit objects, at least 100,000 root tree objects, plus all unique subtrees and blobs. Because Git heavily deduplicates unchanged files, you'd likely visit somewhere between **500,000 to 2 million unique objects**, depending on the repository's churn rate.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

A6.3: GC Race Condition

- The Race Condition**:** 1. You run `pes commit`. The system hashes a modified file and writes Blob X to the object store. 2. Before your commit finishes building its tree and updating `HEAD`, a background GC process kicks off. 3. GC runs its "Mark" phase. Because Blob X isn't linked to `HEAD` or any branch *yet*, GC flags it as unreachable. 4. GC sweeps and deletes Blob X. 5. Your commit process finishes and updates `HEAD`. 6. Your new commit now points to a tree containing a deleted blob, instantly corrupting the repository.
- Git's Solution**:** Git avoids this using a grace period. Real `git gc` checks the filesystem modification time (`mtime`) of unreachable objects. It will only delete an unreachable object if it is older than a specific threshold (typically 2 weeks). Freshly written objects are completely ignored by the garbage collector.